OCTOPORT — ОПТИМИЗАЦИЯ ДИСКА

Полный доказательный аудит и адресный план освобождения 25 ГБ

Технический аналитический отчёт для владельца и исполнителей Octoport

31 октября 2026 года

СОДЕРЖАНИЕ И НАВИГАЦИЯ

Часть I · Что занимает место и почему: 0. Решение владельцу · 1. Методика доказательств · 2. Распределение 56,18 ГБ · 3. Рабочие копии · 4. Зависимости и веб-сборки · 5. Исторические результаты · 6. Отдельные партии.

Часть II · Что конкретно делать: 7. Защищённые релизы и данные · 8. Модель цели 25 ГБ · 9. Очерёдность действий и ответственные · 10. Технические инструкции · 11. Риски и защиты · 12. Условия завершения.

Приложения: A. Первичные файлы, хеши и доказательства · B. Репозиторий и координация · C. Объяснение технических терминов.

Три контрольные цифры

Доказанный проектный расход	Условный базовый резерв	Цель сокращения
56,18 ГБ	**22,736 ГБ**	**25 ГБ**

Важно: 22,736 ГБ — расчёт после проверяемых условий, а не уже освобождённое место. Статус удаления, исходные доказательства и необходимые исключения указаны непосредственно рядом с каждым мероприятием.

OCTOPORT: ПОЛНЫЙ АУДИТ ДИСКА И ПРОГРАММА ОПТИМИЗАЦИИ

Практический документ для владельца, Контроллера, dot, A/B/C и Тестера
Срез данных: 31.10.2026, проверки 04:48–05:26 UTC.
Сервер: Easyscript, Remote Desktop Commander, устройство 8a39166a-4f24-4d75-891f-390a7d7ad62f.
Репозиторий: MaksimUnimax/runtime-fixtures.
Режим документа: план и инструкции; сам документ не подтверждает удаление каких-либо новых каталогов.

> **Главный вывод.** Проектный набор каталогов занимает около **56,18 ГБ**, потому что разработка многократно сохраняет полные рабочие копии, установленные зависимости и распакованные результаты проверок. Измеренный условный резерв сокращения — **22,736 ГБ**, а с отдельно рассмотренным (пока защищённым) сценарием хранения старых релизов — **23,496 ГБ**. Это *не* уже освобождённое место. Для достижения 25 ГБ необходимо дополнительно доказать минимум **1,504 ГБ** чистого сокращения даже в расширенном сценарии.

0. Решение для владельца — на одной странице

Проблема не в том, что исходный код «весит 56 ГБ». Масштаб исходного дерева оценивается десятками мегабайт; десятки гигабайт создаёт **способ организации разработки**. На сервере одновременно хранятся сотни Git worktree, повторяющиеся деревья runtime/extracted, наборы node_modules, .next, старые пакеты и исторические результаты.

Рекомендуемое решение: не удалять историю и WIP, а изменить жизненный цикл временных окружений. Сначала сохранить все уникальные изменения, файлы и доказательства, затем штатно закрыть нетекущие деревья, компактно удерживать завершённые результаты и обеспечить повторное использование *проверенных* неизменяемых входов сборки.

Показатель	Подтверждённое значение	Как трактовать
Атрибутированный проектный расход	**56 177 659 904 Б**	Физические блоки ряда проектных путей, не размер одного приложения
Зарегистрированных Git worktree	**440**	Из них 75 на /dev/shm, один защищённый R3 исключён
Старые по наблюдаемому состоянию копии	**358**, около **14,458 ГБ**	Не означает разрешение удалить все; WIP есть
Модель сокращения с сохранением главных источников	**22 735 521 794 Б**	Условная, после квалификации потребителей
Дополнительный сценарий по релизам	**760 619 008 Б**	Пока **НЕ разрешён**: релизы/откат защищены
Недоказанный остаток до цели 25 ГБ	**1 503 859 198 Б**	Даже если расширенный сценарий реализуем
Реально очищено ранее (две партии кэшей)	**545 431 552 Б**	Результат предыдущей операции, **не** часть новой модели
Свободно на последней контрольной проверке	**7 924 736 000 Б**	На 05:26 UTC; живой сервер, значение меняется

Чего не делать: не выполнять рекурсивное удаление /root/octoport-control, не стирать все worktree подряд, не трогать файлы БД, авторизации и браузерные профили, не переписывать чужой WIP, не объединять изменяемые файлы разных исполнителей жёсткими ссылками. Защищённые Business Bridge, R3, действующие релизы и необходимые точки отката сохраняются.

Приоритет ответственности: организацию закрытия worktree и сохранения WIP выполняют их прежние авторы A/B/C под dot/Контроллером. Дисковый исполнитель отвечает за отдельно переданные сборочные остатки и кэши. Тестер подтверждает свой пиковый бюджет и потребителей. Новый «единый уборщик», демон или параллельный автор не нужны.

---

1. Методика и уровень достоверности

1.1. Что именно измерялось

Суммы приведены **в байтах, фактически выделенных файловой системой**, а не только в логическом размере файла. Измерение использовало `du` в одном проходе для основных корней, `stat/st_blocks`, сведения о `st_nlink`, Git HEAD/ref/status и контрольные хеши. За счёт этого файл, на который ведут несколько жёстких ссылок, не объявлялся несколькими независимыми сотнями мегабайт.

- **ГБ** в данном документе: 1 000 000 000 байт. **ГиБ:** 1 073 741 824 байта. Цель «25 ГБ» трактуется как **25 000 000 000 байт**; если владелец имел в виду 25 ГиБ, целевой бюджет больше.
- `du` для разных поддеревьев, выполненный в разном порядке, может приписать общий inode разным путям. Поэтому размеры родителей и дочерних каталогов **не складывать**. Итоговая модель построена по явно непересекающимся группам.
- Сервер живой: полученные срезы не являются атомарным snapshot. Дополнительные задания могут менять `df` параллельно; в квитанции каждой партии фиксировать собственный прирост до/после, а не обещанный глобальный остаток.
- Для части целей доказано **байтовое совпадение**, но ещё не доказано отсутствие будущего потребителя. Такие цели — кандидаты на адресное закрытие, а не «безусловный мусор».

1.2. Уровни доказательства

Код	Что доказано	Пример	Можно ли сразу удалять?
D1	Физические блоки и владелец каталога	14,458 ГБ нетекущих worktree	Нет
D2	Идентичность файлов / SHA-256 / восстановимость	Архив и распакованный CI-stage	Не без проверки читателей
D3	Все нынешние и плановые потребители закрыты; WIP и ссылки сохранены	Персональная квитанция автора	Возможен адресный акт
D4	Исполнение и readback, df до/после	Ранее очищенные кэши	Это уже фактическая экономия

Текущий статус: большие блоки ниже находятся преимущественно на D1–D2, а не D4. Пустая или завершённая служба сама по себе не доказывает, что её входы больше не нужны для отката, CI или следующего теста.

1.3. Источники

Ключевые материалы — отчёты и машинные первичные инвентаризации, сохранённые на сервере и в GitHub. Точные пути и SHA-256 приведены в приложениях. Исходный состав результатов берётся из сохранённых снимков Octoport, а не из оценок типичного проекта.

---

2. Где именно накоплены 56,18 ГБ

2.1. Крупные корни

Размещение	Физический объём	Содержание / объяснение
/root/octoport-control	**34,130 ГБ**	Главный склад рабочей истории, временных окружений и артефактов
/var/lib/octoport-monitor	**6,767 ГБ**	Ручные тестовые среды, пакеты, зависимости и защищённые данные
/opt/octoport	**4,955 ГБ**	Установленные релизы: не общая корзина для очистки
/root/runtime-fixtures	**0,940 ГБ**	Общий Git, документы и сохранённые воспроизведения
/var/lib/octoport-r142-health-qualification-a1229891a75d824f	**0,821 ГБ**	Квалификационный пакет и окружение
/var/lib/octoport-ci-package-20261030-122100	**0,723 ГБ**	Архив 150 МБ + распаковка 572 МБ

Таблица показывает самые крупные части, **не является полной суммой** всех 1 157 атрибутированных корней. Общая цифра **56,178 ГБ** включает также зарегистрированные рабочие копии вне этих путей и относящиеся к проекту поддеревья /tmp, /var/tmp.

2.2. Внутри главного склада /root/octoport-control

Подкаталог	Байт	Зачем он есть и что проверять
temporary	**12 251 029 504**	Старые копии рядом с необходимыми R3 и PostgreSQL; удалять только отдельные проверенные пути
worktrees	**8 973 201 408**	Многочисленные Git-копии A/B/C/Контроллера
logs	**6 448 926 720**	Большинство — не обычные .log, а runtime/extracted/ZIP и результаты
controllers	**1 549 787 136**	Копии проверок, состояния и публикационные результаты
browsers	**1 139 662 848**	Бинарники/профили; отделять одно от другого
design	**1 067 855 872**	Изображения, исходники и браузерные данные
b01	**679 075 840**	Старое окружение с БД — **KEEP**
artifacts	**423 768 064**	Снимки и результаты, в том числе C

Особо защищённые элементы внутри temporary: **R3 5 418 569 728 байт**, retained-postgres-20261002 **1 744 826 368 байт**. Они **не относятся к разрешённому сокращению**. Каталог dot-extension-20261006T0254Z занимает порядка **1,541 ГБ**, однако там есть сохранённые браузерные профили и снимки; удаление всего каталога недопустимо.

2.3. Почему «маленький продукт» превращается в десятки гигабайт

Для кода в Git характерен масштаб десятков мегабайт. На сервере же хранятся **полные распакованные копии той же версии**: исходник, отдельная изменённая копия, составленный runtime, извлечённый пакет, отрицательная тестовая сборка и несколько исторических выпусков. Каждая следующая проверка может повторять несколько стадий вместо использования уже сохранённых неизменных входов. Отсутствие правила «проверка закончена → данные классифицированы → собственный временный ресурс закрыт» превращает каждую успешную попытку в постоянное хранение.

Корневая причина — lifecycle, не размер исходного кода или необходимость удалять Git-историю.

---

3. Рабочие копии: максимальный структурный резерв

3.1. Инвентаризация

Обнаружено **440 зарегистрированных Git worktree**. Из них **75 находятся на /dev/shm** (не корневой диск); один защищённый R3-worktree исключён. Проверены **364** остальные дисковые рабочие копии и **692 202 существующих отслеживаемых файла**. Их отслеживаемые исходники занимают **12 269 199 360 байт**.

Группа расположения	Копии	Физические байты отслеживаемых файлов
A	53	1 844 408 320
B	57	1 766 674 432
C	143	4 374 335 488
Контроллер	16	492 142 592
Прочие зарегистрированные места	95	3 791 638 528

358 копий не были отмечены как текущие в проверенных состояниях и cwd. Их общий физический размер — **14 458 081 280 байт**. Это адресный массив для разбирательства прежними авторами, а не разрешение удалять 358 каталогов.

HEAD всех 364 проверенных деревьев найдены среди **5 190 коммитов, достижимых из локальных ссылок Git**. Это не доказывает, что все они опубликованы на GitHub: перед закрытием конкретной копии нужно проверить локальную ветку, удалённый ref или закрепить сохранённый commit идентификатор.

3.2. Первая волна: 128 условно чистых деревьев

Выделено **128 копий** без видимых tracked/untracked изменений, отсутствующих файлов, специальных skip/assume-флагов, Git-lock и обнаруженного текущего потребителя. Их tracked-исходники — **4 194 750 464 байта**.

Что мешает удалять их немедленно: могут существовать **игнорируемые Git результаты**, ещё назначенные будущие проверки, ссылки архивов на исходный путь или открытые PR. Часть деревьев не имеет отдельной локальной ветки на HEAD — достижимость коммита из других refs не равно персональной сохранности. Для каждого дерева автор закрывает эти риски и подтверждает результат.

Конкретные примеры из полной машинной ведомости (без разрешения на удаление от одного только попадания в таблицу):

Путь, относительно /root/	Tracked-блоки	HEAD (первые 12)
octoport-control/worktrees/a01-pr354-exact444-terminal-review-20261028	43 872 256	444a2dd739e2
octoport-control/worktrees/a01-pr354-exactd45-midbody-consumer-20261028	43 851 776	d45ebdf3ffa8
octoport-control/worktrees/a01-pr354-exact236-vault-review-20261028	43 831 296	236c6bd106a9
octoport-control/worktrees/a-c04-inbound-journal-exact7f-20261029	42 713 088	3323b0a0ee3d

Путь, относительно /root/	Tracked-блоки	HEAD (первые 12)
octoport-control/worktrees/a03-opera136-alice-a26-20261027	42 479 616	ecb56f62ba6d
octoport-control/temporary/C/c03-openapi-param-override-r1-20261012	39 231 488	ef7881216d3b

Полный список всех 128 с HEAD, ветками и размерами: /root/octoport-control/temporary/disk-owner/DISK_OWNER_20261031_OPTIMIZATION25_SUMMARY.json (SHA-256 ceb2029a2cbbfdc0665350f7e6182cf9b0d1d0ec460e119a6b35069c0807cfc5).

3.3. Ещё 221 дерево: гигабайты исходников, мегабайты уникальной работы

У 221 старой копии с tracked WIP или untracked-файлами проверено:

- Полный развёрнутый tracked-код: **7 525 593 088 байт**.
- Полный бинарный diff против HEAD: **42 441 196 байт**.
- Измеренное сжатие этих diff-потоков в памяти: **8 714 238 байт**.
- Untracked: **69 097 316 логических байт**, или **72 765 440 физических байт**.

То есть *основной объём здесь — много раз разложенный код, уже имеющийся в Git*. Но **8,71 МБ — не готовый сохранённый комплект WIP**. Для каждого дерева нужны исходный HEAD/ref, staged/unstaged изменения, untracked, нужные ignored, права/ссылки и доказанное восстановление. Только после этого можно освободить полную копию.

Имеются **32 828 путей, указанных Git, но отсутствующих в соответствующих рабочих каталогах**. Они не занимают физического места; при сохранении состояния нужно различить намеренные удаления, sparse checkout и специальные Git-флаги. Не считать это «потерянными файлами», которые надо удалять второй раз.

3.4. Конкретный порядок безопасного закрытия одного worktree

Ответственный: текущий автор дерева, при организации Контроллером/dot. **Не:** отдельный дисковый исполнитель, который не владеет WIP.

1. Выбрать **один точный путь** из ведомости; записать автора, цель, HEAD, ветку и открытый PR/потребителей. Проверить незавершённые unit, процессные рабочие каталоги, Git locks, ожидающие задачи и текущий state.
2. Зафиксировать `git status` **с** untracked и ignored, staged/unstaged diff, deletion/sparse-состояние, nested repository, результаты и локальные неподключённые артефакты. Привязать к конкретному commit и исходной версии.
3. **Чистое дерево:** подтвердить, что нет нужных ignored данных, дальнейших потребителей и неподписанных результатов. Убедиться, что HEAD остаётся достижим через стабильную ссылку и при необходимости закрепить ref без перезаписи другой ветки.
4. **Дерево с WIP:** автор сохраняет собственный commit/ветку или полный точный recovery-пакет: diff + untracked + нужные ignored + метаданные. Проверяет восстановление в изолированном месте/логически и фиксирует хеши; нельзя довольствоваться одним текстовым patch для бинарных и новых файлов.
5. Выполнить **штатное удаление конкретного worktree без `--force`**, когда Git и автор подтвердили готовность. Не удалять общую `.git`, refs и ветку только ради экономии.
6. Сверить исчезновение только выбранного пути, восстановимость commit/WIP, `df` и inode до/после, итог в существующем checkpoint. Если Git отказывает в закрытии из-за остаточных данных — **KEEP и разбор**, не обход флагом `--force`.

Рекомендуемый стандарт: у каждого потока иметь известное число актуальных рабочих деревьев и отдельную обоснованную точку отката; старые копии закрывать по окончании задач. Это целевая политика, а не ретроактивное основание удалять неизвестные каталоги.

---

4. Зависимости и веб-сборки: условный резерв 3,871 ГБ

4.1. Результат инвентаризации

В **377 неперекрывающихся корнях** обнаружено 4 235 производных каталогов. Вне отмеченных текущими рабочих копий собственные физические блоки распределены так:

Категория	Физические байты	Риск и смысл
node_modules	**4 415 864 832**	Восстановимые *только при подтверждённых* версиях/lockfile/offline inputs
.next	**1 406 459 904**	Здесь не только cache, но и готовые server/build outputs
Python __pycache__ / .pyc	**20 684 800**	Обычно регенерируемые файлы, но конкретный потребитель проверяется
Результаты тестов из этой категории	**126 976**	Уникальные результаты могут быть нужны
Всего	**5 843 136 512**	Это физический объём, не весь безопасный объём удаления

Из него **1 971 716 096 байт** консервативно оставлены: донорские зависимости, rollback-окружения, квалифицированный пакет и другие явно удерживаемые источники. **Остаток модели — 3 871 420 416 байт.** Внешние hardlinks ещё на 533 037 056 байт не объявлены освобождаемыми.

Текущий `/root/octoport-c03-receipt-count-20261029/node_modules` (~550,56 МБ) активен и **исключён**. Также не трогать текущие установки Node22/Node24, `_npx`, RDC, браузерные профили и опубликованные релизы.

4.2. DEP-01: три адресных `node_modules`, восстановление уже доказано

Все три находятся под одним корнем:

`/var/lib/octoport-monitor/health-manual-20261025/b02-r92-installed/`

Удаляемый дочерний каталог	Собственные блоки, Б	Сохраняемое основание восстановления
r142-server-package-source-eb206822/node_modules	558 260 224	r142-server-package-source-b2bdbf27/node_modules + точные различия
r142-server-package-source-ba661435/node_modules	558 239 744	Тот же донор + точные различия
r142-server-package-source-2d759a19/node_modules	550 256 640	r142-exact-source-09548cb0/node_modules
Всего DEP-01	**1 666 756 608**	Входит в 3,871 ГБ выше; **не прибавлять вторично**

Доказательство: для всех пяти деревьев посчитано по **27 819 записей** с проверкой байтов, symlinks и прав. У двух целевых деревьев по 37 небольших файлов отличается от донора; все **74 файла**, включая Vitest results, сохранены. Для третьей цели донор полностью совпадает. Хеши соответствуют прежнему manifest. Пакет восстановления размером **186 050 байт** лежит на Easyscript:

/root/octoport-control/temporary/disk-owner/
DISK_OWNER_20261031_R142_DEPENDENCY_RECOVERY.json

SHA-256: 0c439ffa2c5fdafe8323ab3dda13992125941397d67b2b9620d48b01560750ae.

Что уже произошло: владелец дал конкретное разрешение DEP-01. Исполнитель проверил пути, исходники и остановленные старые сборки, но команда адресного удаления получила **платформенную блокировку до запуска**. Повторная проверка показала, что все три каталога сохранены. **DEP-01 фактически освободил 0 байт**. Нельзя выдавать permission за выполнение или заменять заблокированную операцию обходом через другой маршрут. Потребительская передача должна оставаться подтверждённой.

Как завершать DEP-01 после разрешения блокировки: сверить идентичность inode/HEAD/lockfile и два сохраняемых донора → получить подтверждение об окончании использования → адресно выполнить удаление **только** этих трёх node_modules в безопасном инструментальном контексте → сохранить readback и разницу df → проверить доступность донорских путей и receipt. Ничего в родительских директориях, .git, apps, package ZIP, профилях или БД не удалять.

4.3. Отделение .next/cache от готовой сборки

На практике в apps/admin/.next и apps/portal/.next лежат разные классы данных. Кэш Next.js (.next/cache) можно регенерировать при следующей сборке; серверные выходы .next/server, manifests и другие файлы могут входить в готовый пакет и систему отката.

Порядок для каждого каталога: определить точный исходный SHA и завершённую попытку → проверить, не входит ли файл в RELEASE_SHA256SUMS → убедиться, что сборка/тест не выполняется и пакет уже сохранён → измерить исключительно .next/cache → адресно убрать только cache → проверить, что готовая сборка и её SHA остались неизменными. **Запрещено трактовать все 1,406 ГБ .next как удаляемый кэш.**

4.4. Как не накапливать node_modules вновь

Сценарии 0707Z-stage-build.py и 0711Z-stage-build.py использовали git clone --no-hardlinks, shutil.copytree хранилища пакетов и pnpm --package-import-method=copy. Это делает **новые физические копии** даже для повторов того же SHA. Постусловие finally сохраняло результат, но не завершало lifecycle полного окружения.

Нужное исправление действующего entrypoint (не косметика исторических файлов):

1. До любой тяжёлой копии проверить точные версии Node/pnpm/Python, SHA lockfile, источник SHA, доступность **всех** offline dependencies и бюджет максимального прироста. Не устанавливать зависимости ради одного лишь такого preflight.
2. Вычислить идентификатор неизменяемых входов из (source SHA, tree, lockfile SHA, Node/pnpm, platform, build mode, verified package inputs); использовать его в уже существующем checkpoint. Не вводить дополнительный глобальный реестр или daemon.
3. Если уже есть **собственное завершённое** окружение точно тех же входов, проверить его целостность и отсутствие другого автора, затем использовать повторно — только когда сценарий допускает это. Если ожидается модификация содержимого, создать изолированную частную копию изменяемой части.
4. Не копировать весь pnpm store в каждую попытку по умолчанию. Предпочитать проверенные неизменяемые blob-источники из существующего хранилища; разрешённый метод импорта выбирать

после проверки файловой системы и поведения тестов. **Не делать общие изменяемые hardlinks между потоками.**

5. Для необходимой полной копии учитывать **фактический физический пик**, включая параллельно удерживаемые результаты, а не только ожидаемый размер ZIP.
6. В конце попытки сохранять один точный пакет, полный результат, SHA и нужные уникальные evidence. В собственном checkpoint отмечать закрытые временные каталоги, а не оставлять полный store после каждого FAIL/PASS.

Проверка исправления: только на следующей действительно необходимой продуктовой сборке: свободные байты/inode до начала, максимум фактически занятого места в процессе, точный результат сборки, объём сохранённых результатов, место после корректного завершения; сверить, что не возникла очередная полная копия при неизменных входах. Одна правка кода без такого запуска — **не доказательство устранения проблемы**.

---

5. Исторические результаты: экономить 5,765 ГБ без потери доказательств

5.1. Что именно лежит в logs и controllers

В этих каталогах хранятся не только текстовые логи. Результаты тестов включают распакованные расширения, исходники, отрицательные тестовые фикстуры, повторные ZIP, скриншоты, JSON и полный программный runtime. Именно здесь ранее обнаружено **39 770 дополнительных идентичных физических копий**. Для соответствующих непересекающихся диапазонов размеров подтверждена SHA-256 избыточность **2 872 582 144 байта**.

Но более перспективная стратегия — **компактное хранение завершённых групп результатов**: она охватывает и идентичные файлы, и уникальные, поэтому не теряет доказательную полноту. Ранее выявленные 2,873 ГБ дубликатов **нельзя прибавлять** к модели сжатия ниже.

5.2. Проверенное сжатие

Выбраны **146 836 обычных файлов старше суток** из 1 707 групп. Из анализа исключены или сохранены отдельно профили, приватные файлы, базы, R3, Business Bridge, Git, зависимые активные пути и иные защищённые элементы.

Показатель	Байты
Текущие собственные блоки выбранных файлов	**8 126 296 064**
Эквивалентное размещение измеренного tar+gzip -1	**2 361 667 584**
Расчётное сокращение файловых блоков	**5 764 628 480**

Потоки архивов сжимались **в памяти**, затем распаковывались обратно; SHA-256 полного tar-потока совпадал, ошибок измерения не обнаружено. **Постоянный архив на сервер вместо исходников ещё не записывался**, его физическое размещение, полная метаинформация и конечная читательская совместимость пока не проверены.

Самые крупные конкретные группы:

Группа	Выбранные исходные блоки	Архивная оценка	Сокращение
controllers/audits	1 126 014 976	391 569 408	**734 445 568**
logs/B	387 190 784	87 535 616	**299 655 168**
logs/A	351 059 968	88 776 704	**262 283 264**

Группа	Выбранные исходные блоки	Архивная оценка	Сокращение
logs/C	339 492 864	88 780 800	**250 712 064**
artifacts/C (выбранные файлы)	272 318 464	55 959 552	**216 358 912**

Это размеры **выбранных старых файлов**, а не полные размеры каталогов. Не архивировать поверх живых файлов, не удалять исключённые элементы и не предполагать, что каждый читатель умеет работать с .tar.gz.

5.3. Как выполнять компактное хранение одной группы

Ответственный: автор/держатель исторической evidence-группы; согласование — Контроллер. **Цель:** сохранить *всю* уникальную информацию, перестав занимать гигабайты распакованным деревом.

1. Выбрать конкретную завершённую группу, сохранить список всех дочерних файлов, точных хешей, размеров, modes, uid/gid, относительных symlinks, пустых каталогов, возможных xattrs и связей с первоначальным checkpoint. Зафиксировать файлы, которые **не должны** быть перемещены.
2. Установить читателей: текущая задача, открытый PR, README/receipt, исторический audit, автоматизированная проверка, предусмотренный откат. Если читатель требует исходный путь и не умеет восстановить архив, группу оставить на месте.
3. Выполнить preflight по месту: свободно ≥ обязательный минимум + действительные резервы + **размер будущего архива и пиковые временные буферы**. Архив создавать *по одной группе*, не 2,36 ГБ за один раз на почти заполненном диске.
4. Записать архив в заранее согласованный путь в уже существующей структуре артефактов; использовать формат с воспроизводимым порядком и сохранением метаданных. Обязательно получить полное успешное завершение записи/fsync.
5. Проверить SHA архива, список членов, SHA содержимого, symlink/mode/uid/gid и выборочное реальное восстановление там, где допускает место. Отдельно проверить отсутствие недостающих файлов и невозможность path traversal при распаковке.
6. Обновить существующий checkpoint/receipt: канонический архив, SHA-256, список исходных путей, процедура чтения и результат проверки. Убедиться, что все нужные потребители теперь читают архив или получают разрешённое восстановление.
7. Только после выполнения всего выше и адресного подтверждения удалить **уже освобождённые исходные файлы**; не выполнять рекурсивное удаление родителя с защищёнными записями. Измерить df/inode. Отрицательный результат проверки — **STOP/KEEP**, не force-переименование.

5.4. Требование к новым результатам

Новый запуск должен оставлять **один точный ZIP/tar с SHA**, исходный SHA и краткий receipt; другая проверка того же идентичного пакета ссылается на них, а не снова дублирует байты. При этом сохраняются результаты тестов, которые реально отличаются, а негативные или мутирующие тесты по-прежнему получают изолированную копию. Не заменять пакеты общей изменяемой папкой: это создаст гонки и разрушит воспроизводимость.

---

6. Небольшие, но очень хорошо доказанные отдельные партии

6.1. Распакованный CI-stage — 572 489 728 байт

Точный кандидат (не исходный worktree):

/var/lib/octoport-ci-package-20261030-122100/owner-test-runtime-a1229891a75d824fb89ead6722025ceb2b135119/a1229891a75d824fb89ead6722025ceb2b135119

Сохраняемый файл: в том же родителе octoport-owner-test-a1229891a75d824fb89ead6722025ceb2b135119.tar.gz, **150 262 103 байта**, SHA-256 1b46df98aeeddb86c93c688788029013bf876da710bc5d9b2ad884d915830061. Он уже указан в исходном PACKAGE_RECEIPT.json.

Проверка потока архива без записи распаковки: **16 567 файлов, 2 088 каталогов, 302 symlink**; полный набор путей, modes, ссылки и содержимое совпали; mismatches 0. Сборочный unit завершён успешно, активный процесс не найден. **Это сильная D2-цель; для D3 всё ещё нужны будущие потребители и роль в откате.**

Порядок: сохранить архив+receipt+лог неизменными → подтвердить отсутствие следующей запланированной проверки и необходимого rollback → повторить manifest/identity перед действием → адресно закрыть *только распакованный внутренний каталог* → контроль df и SHA архива. Восстановление — повторное извлечение tar.gz, **не новая сборка**.

6.2. Вторая копия браузерных бинарников — 408 018 944 байта

Кандидат: /tmp/octoport-r142-public-cft151.
Сохраняемая дистрибуция: /var/lib/octoport-monitor/health-manual-20261025/b02-r117-cft151-owned-resources.

Проверено **303 файла / 316 записей**; байты совпадают. SHA самого chrome в обоих деревьях: 0b20b130e7edd9dd51873be867761295fe0cfad490c2b9a64f95bd3cfc08fa71.

Важная оговорка: файловые права и владельцы *разные*: первая копия root/root, вторая octoport-monitor; 22 записей с режимами 0755/0700, 294 — 0644/0600. Поэтому обычный symlink между security contexts **не рекомендован**. На сервере сохранена точная карта metadata. Перед закрытием /tmp-дистрибуции нужно удостовериться, что тесты больше не обращаются именно по старому пути; при необходимости корректно изменить путь в *собственном* тестовом scope, не меняя защищённые профили.

6.3. Сжатие закрытых журналов инструмента — 388 595 712 байт

В /root/.claude-server-commander/ есть **49 закрытых датированных журналов** общей физической ёмкостью **517 939 200 байт**. Измеренное gzip-сжатие в памяти — ~129 343 488 байт физических блоков. Для каждого журнала выполнена распаковка и SHA-проверка без вывода содержимого. **Активный** claude_tool_call.log исключён.

Предложение: проверить штатного читателя архивов, сохранить исходный хеш/дату, сжать закрытые файлы по одному, проверить roundtrip и только затем заменить соответствующий *закрытый* оригинал. Не удалять логи, скрывающие результаты работы; не менять текущую конфигурацию аудита без владельца журнала.

---

7. Установленные релизы, общие инструменты и защищённые данные

7.1. Релизы нельзя оценивать по независимым du

В /opt/octoport/ops-releases сохранено несколько версий. При проверке **305 валидных systemd unit** найдены **шесть различных SHA, закреплённых действующими или предусмотренными службами**, включая служебную пару octoport-monitor-retention и octoport-monitor-retention-inspect. Значит, проверить только шесть наиболее видимых продуктовых процессов недостаточно.

Для части иных релизов рассчитано сохранение собственных файлов в архивном виде: **положительный условный выигрыш 760 619 008 байт**. Однако несколько релизов имеют много общих жёстких ссылок: у каталога с 46 МБ собственных блоков автономный архив может весить 223 МБ. **Компактное хранение не всегда экономит место.**

Решение: держать все действующие и необходимые откатные релизы. Неприкреплённые варианты рассматривать только после отдельной матрицы «служба — pinned SHA — rollback — manifest — потребители», с предварительным созданием и проверкой восстановимого пакета. Эти **0,761 ГБ не включены в базовый разрешённый план**.

7.2. Codex 0.165.0: не удалять по одному факту новой версии

В профилях codex2, codex3, codex4 текущая версия 0.166.0, но валидный selection-history закрепляет 0.165.0 как **previous**. Штатная политика pruning этой версии сохраняет previous для отката. Ранее предложенные ~1,338 ГБ **не являются доказанным мусором**. Авторизации, конфигурации, SQLite, sessions и профили сохраняются.

7.3. Что находится вне этой программы

Защищённый объект	Почему KEEP
Личный пакет R3 (~5,419 ГБ)	Особая сохранённая работа и входы, отдельный запрет
retained-postgres-20261002 (~1,745 ГБ)	База/точка восстановления; не кэш
Business Bridge (все версии, backups и состояние)	Явно исключён владельцем
Авторизованные браузерные профили, cookies и private bindings	Повторное получение авторизаций может быть невозможно
Не завершённые WIP/тесты, текущая копия Тестера	Потеря исходников/доказательств и риск второго автора
Установленные сервисные релизы и необходимые rollback	Нарушение восстановления и работы сервисов
ОС, /swap.img, Docker/containerd вне доказанного scope	Системные обязанности, не «мусор Octoport»

Не использовать чистку операционной системы как способ достичь красивого числа 25 ГБ. Работать с причиной накопления проекта.

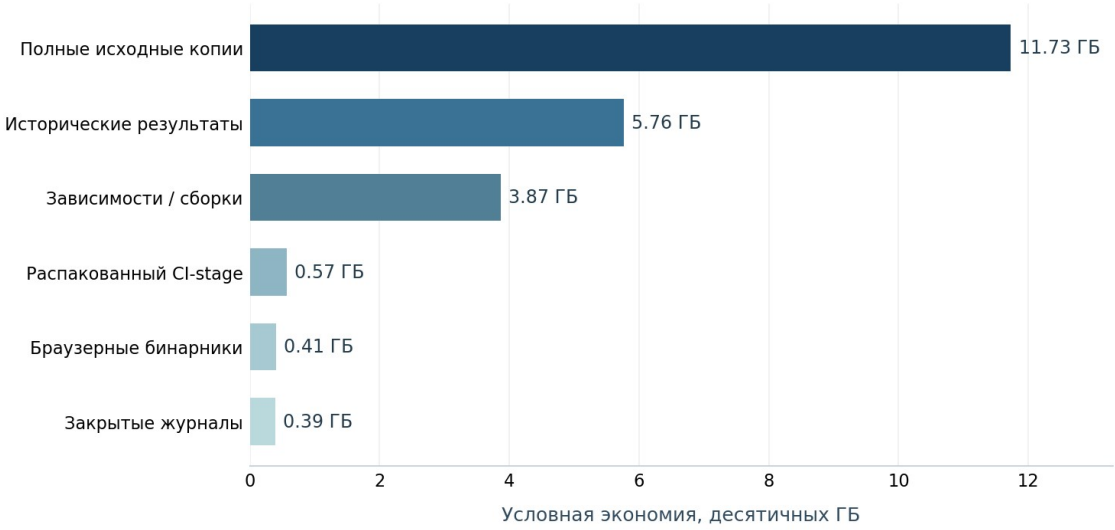
---

8. Финансовая модель места: сколько можно получить без двойного счёта

8.1. Базовый расчёт

Мера	Чистый условный потенциал, Б	Откуда число
Убрать старые полные source-копии, сохранив измеренные diff	**11 730 368 514**	11 739 082 752 собственных tracked-блоков минус 8 714 238 сжатых изменений

Мера	Чистый условный потенциал, Б	Откуда число
Зависимости и web outputs после консервативного KEEP	**3 871 420 416**	5 843 136 512 минус 1 971 716 096 сохранённых входов
Завершённые исторические результаты — lossless retention	**5 764 628 480**	8 126 296 064 минус 2 361 667 584
CI-stage при сохранении verified tar.gz	**572 489 728**	Точные собственные блоки внутренней распаковки
Дублированный Chrome for Testing	**408 018 944**	Точные собственные блоки root-копии бинарников
Сжатие 49 закрытых журналов инструмента	**388 595 712**	Измеренный физический размер до/после модели gzip
БАЗОВАЯ МОДЕЛЬ	**22 735 521 794**	**22,736 ГБ**
Дополнительно: неподтверждённое удержание релизов	**760 619 008**	Отдельный защищённый сценарий
Модель с релизным сценарием	**23 496 140 802**	**23,496 ГБ**
Дефицит до 25 000 000 000 Б	**1 503 859 198**	Даже в расширенном сценарии



Структура условного потенциала сокращения по мероприятиям

Как правильно читать эту таблицу: это оценка *экономии после полного выполнения всех условий*. В частности, исходные WIP, ignored, архивные метаданные, расход на квитанции, вынужденные временные копии и активные потребители могут уменьшить итог. source не пересекается с отдельно измеренными derived-категориями и выбранной исторической областью, но перед каждой физической операцией нужен новый расчёт по inode. **Не прибавлять 14,458 ГБ общего объёма нетекущих worktree к строке 11,730 ГБ: это родительская величина.**

8.2. Что НЕ прибавлять к базовой модели

- **2,873 ГБ SHA-256 дубликатов** уже содержатся в более полной стратегии сохранения исторических результатов (5,765 ГБ), поэтому являются **альтернативной** экономией, а не дополнительной.
- **1,667 ГБ DEP-01** уже входят в derived-пул 3,871 ГБ.
- **4,195 ГБ 128 «чистых» worktree** уже входят в source-модель 11,730 ГБ.

- **545,432 МБ прежней очистки кэшей** — измеренный результат предыдущих операций, а не будущий резерв. Его не прибавлять к текущей потенциальной экономии, чтобы не смешивать уже очищенное с условным.
- Размеры родительских папок, общий `du` и размер файлов с несколькими hardlinks не складывать независимо.

8.3. Как найти недостающие минимум 1,504 ГБ

Пока не доказано, что после перечисленных допустимых оптимизаций обязательно удастся освободить все 25 ГБ. Дополнительное исследование должно оценивать остаточные блоки, а не назначать цель задним числом.

Приоритет дальнейшего поиска:

1. **Остаток нетекущих worktree:** после вычитания известных tracked и derived-блоков остаётся около **1,214 ГБ неразобранной разности**. Она включает каталоги, ignored/уникальные данные, возможные shared-блоки. Не считается дополнительной экономией, пока не построена отдельная inode-карта и авторы не сохранили содержимое.
2. **Старые `/var/tmp` rehearsal/debug** (общая `/var/tmp` в одном проходе ~0,695 ГБ) — проверять точные terminal unit, манифесты и rollback, не всю папку. Часть может уже входить в проектную атрибуцию, поэтому экономию считать только по новому независимому inode-списку.
3. **Отдельные браузерные бинарники в dot-extension:** два каталога браузера около 419 и 408 МБ, но соседние защищённые профили. Только бинарники при доказанной идентичности и сохранении возможности запуска; потенциальный выигрыш пока **не установлен**.
4. **Неиспользуемые staged packages и старые CI outputs за пределами обследованного списка:** искать по завершённым unit и полным manifest, а не по маске «старше N суток».
5. **Релизы — последняя мера:** только если согласованы rollback и применимость содержания; защищённый статус по умолчанию KEEP.

Если после безопасных проверок дополнительный резерв не подтверждается, честный результат — сокращение порядка 22–23 ГБ с отдельным запросом на решение по дальнейшей цели, а **не удаление защищённого R3, DB или истории** ради выполнения плана.

8.4. Ресурсный бюджет после оптимизации

Контроль на 31.10.2026, 05:26 UTC: свободно обычному учёту **7 924 736 000 байт** и **3 317 322 inode**. Обязательный свободный резерв — **3 072 МиБ = 3 221 225 472 байта** и **200 000 inode**. Следовательно, до вычета других обязательств остаётся **4 703 510 528 байт**. Текущее число меняется и должно пересчитываться на каждом допуске.

Формула допуска нового блока:

```
[ = - - ]
```

Для Тестера нужен **полный пиковый прирост всей следующей задачи**, включая параллельные временные файлы и сохраняемые результаты, а не произвольные «5 ГБ». В старом учёте 92 OPEN/HELD с суммой **21 523 МиБ**, но это сумма записей, а не доказанная сумма ещё не реализованных расходов. Их следует адресно сверить с материализованными путями и закрытыми задачами; **нельзя просто обнулить резервы**.

---

9. Исполнимый план: очередность, владельцы, проверки и эффект

Этап 0. Зафиксировать базу и границы

Владелец исполнения: Контроллер/dot вместе с дисковым исполнителем.
Результат: одна актуальная карта авторов и потребителей, без новых worktree и без перезаписи старых checkpoints.

- Сверить текущий deviceId, df -B1 -T /, df -i /, шесть служб и отдельный monitor-retention unit. Сохранить время и хеш реестра исходных показаний.
- Для каждого из A/B/C, Тестера и Контроллера прочитать текущий state и active unit, отметить sole writer каждого затрагиваемого пути. Операцию с неизвестным исходом не повторять.
- Согласовать действительную будущую часть OPEN/HELD; записать не только «зарезервировано», но и какие байты уже материализованы.
- Убедиться, что R3, Business Bridge, БД, браузерные профили, авторизации и pinned releases остаются вне операций.

Условие выхода: каждому выбранному пути сопоставлены точный владелец, текущее использование, ожидающие проверки, восстановительные источники и решение KEEP/RETIRE/WAIT. Без такого списка массовая очистка недопустима.

Этап 1. Первая очередь полностью восстанавливаемых одиночных остатков

Кандидаты: CI-stage **572,49 МБ**, дубль Chrome-бинарников **408,02 МБ**, закрытые журналы **388,60 МБ**.
Условный верх этих трёх мероприятий — **1,369 ГБ**, фактический эффект нужно измерять отдельно.

Действие	Исполнитель	Gate перед освобождением	Способ возврата
Закрыть внутренний CI-stage a122989…	Владелец законченной CI-сборки + дисковый исполнитель	Архив/receipt SHA, нет будущего читателя, нет rollback-зависимости	Извлечь существующий tar.gz и проверить manifest
Закрыть /tmp/octoport-r142-public-cft151	Владелец browser-тестов + дисковый исполнитель	Все пути потребителей заменены/освобождены; права другой дистрибуции учтены	Копия из сохранённого бинарного источника с первоначальными mode/uid/gid
Сжать закрытые claude_tool_call_*.log	Владелец журнала/инструмента	Поддерживается чтение .gz, исходные hashes сохранены	Обратная распаковка gzip, проверка SHA

Этап 2. DEP-01 и остальные доказанно завершённые зависимости

Потенциал текущей модели: 3,871 ГБ с учётом всех KEEP; здесь **DEP-01 составляет 1,667 ГБ**. Начать с независимых подтверждённых целей, не перезаписывать старый платформенный отказ и не менять инструменты ради обхода запрета.

Gate: terminal build/status, source HEAD, lockfile, подтверждённые доноры/offline inputs, владелец/плановый потребитель, точный путь. Для текущих веб-релизов cache и готовая сборка должны быть явно разделены. После партии — новые df и inode, сохранённые исходники/архивы и квитанция.

Этап 3. Сокращение worktree без потери WIP

Потенциал source-модели: 11,730 ГБ. **Исполнители:** исходные A/B/C и Контроллер. Начать со **128** наиболее чистых кандидатов; затем обрабатывать **221** дерево с WIP небольшими, подписанными партиями. Включать адресный readback Git refs и персональных изменений.

Массовое использование git worktree prune или rm -rf /root/octoport-control/worktrees запрещено как замена анализу: это может уничтожить уникальные ignored и сломать активную работу. Даже «чистые» 128 кандидатов должны пройти проверки из §3.4.

Этап 4. Компактное хранение исторических результатов

Потенциал: 5,765 ГБ без уничтожения байтов. **Исполнители:** владельцы evidence-групп и Контроллер. Выполнять группами, сначала несколько завершённых старых наборов с минимальным числом читателей. После каждого архива выполнить SHA/metadata/restore/read-path gate, затем освобождать только согласованные исходные пути.

Важно: этот этап вместо duplicate-only массового dedup; не объединять его с параллельным удалением отдельных совпавших файлов до завершения картирования.

Этап 5. Исправить генераторы и закрытие результатов

Результат: источник расхода устранён в реально используемом entrypoint и проверен на следующем нужном продуктовом запуске.

1. Назначить одного автора конкретных tooling/build/*, tooling/checks/* и R142-driver, получив фактическую передачу соответствующих путей.
2. Вынести проверки инструмента/lockfile/offline input перед clone/copy; выполнять их без выделения тяжёлого диска.
3. Обеспечить reuse точно совпадающих immutable inputs в собственном scope. Хранить один канонический пакет с SHA, а не повторять полные физические snapshots.
4. Пересмотреть extension_composed.build: сохранить проверки воспроизводимости, но не оставлять несколько полных деревьев после их последнего читателя. Негативные тесты и mutable data — строго изолированы.
5. В существующих result/checkpoint отметить temporary paths и факт RETIRED только после окончательного закрытия. Не создавать отдельную систему хранения.
6. Применить новый порядок на следующей необходимой сборке и измерить до / пик / после; корректность результатов и SHA не должны измениться.

Этап 6. Добрать недостающий резерв только после первых партий

Построить inode-карту остатка worktree и отдельных старых rehearsal environments. Если после реальных замеров достигнута требуемая ресурсоёмкость Тестера и оставлены обязательные резервы — **не ждать всей исторической чистки ради продолжения продукта**. Если всё ещё требуется именно 25 ГБ, подтвердить дополнительную уникальную экономию минимум 1,504 ГБ, не списывая незапланированные защищённые данные.

---

10. Операционные инструкции для исполнителей

Данный раздел предназначен для технического исполнителя; владельцу достаточно контролировать итоговые «до/после» и сохранность работы. Примеры команд — **диагностика и шаблон процедуры**, а не готовый скрипт массового удаления. Значения $WT, $OUT, $TARGET подставлять только из заранее согласованной адресной партии.

10.1. Предварительный read-only контроль

```
# Текущее время и реальное место, все значения в байтах.
date -u '+%Y-%m-%dT%H:%M:%SZ'
df -B1 -T /
df -i /

# История worktree, без изменения индекса/веток.
GIT_OPTIONAL_LOCKS=0 git -C /root/runtime-fixtures worktree list --porcelain

# Статус ИМЕННО выбранной рабочей копии.
GIT_OPTIONAL_LOCKS=0 git -C "$WT" rev-parse HEAD
GIT_OPTIONAL_LOCKS=0 git -C "$WT" status --porcelain=v1 \
  --untracked-files=all --ignored=matching
GIT_OPTIONAL_LOCKS=0 git -C "$WT" ls-files -v
```

Точка остановки: обнаружен активный автор, Git-lock, неожиданный HEAD, изменившийся inode, UNKNOWN операция, непроверенный ignored-файл или отсутствие подтверждения отката. Состояние сохраняется и передаётся текущему владельцу. Не переходить к force-удалению.

10.2. Сохранение WIP для одного конкретного дерева

```
# Выполняет ИСХОДНЫЙ автор дерева в собственном согласованном scope.
# Пример записи диагностических исходных данных в подготовленный каталог.
GIT_OPTIONAL_LOCKS=0 git -C "$WT" rev-parse HEAD > "$OUT/base-head.txt"
GIT_OPTIONAL_LOCKS=0 git -C "$WT" diff --binary HEAD > "$OUT/tracked.patch"
GIT_OPTIONAL_LOCKS=0 git -C "$WT" ls-files --others \
  --exclude-standard -z > "$OUT/untracked-names.nul"
GIT_OPTIONAL_LOCKS=0 git -C "$WT" status --porcelain=v1 \
  --untracked-files=all --ignored=matching > "$OUT/full-status.txt"
```

Эти команды НЕ сохраняют самих untracked/ignored файлов. После них автор отдельно архивирует и хеширует их исходные байты, проверяет специальные Git-состояния, подпроекты, symlinks и необходимые файлы игнорируемых сборок. Требуется успешное восстановление на сохранённой базе. Нельзя считать tracked.patch самодостаточной резервной копией.

Штатное удаление **после** успешного подтверждения автора выполняется через git worktree remove **без --force**, по одному точному пути. Удаление refs, веток и git gc --prune не входит в эту операцию. Результат должен быть воспроизводим по сохранённому commit и файлам, включая единственные опубликованные/неопубликованные изменения.

10.3. Рецепт хранения завершённой evidence-группы

Выполнять по **одной** заранее выделенной группе под владельцем соответствующего участка. Не выбирать целую /root/octoport-control/logs в качестве $ROOT. Пример полезных проверок:

```
# Пример контроля уже записанного архива, без удаления оригиналов.
sha256sum "$ARCHIVE"
gzip -t "$ARCHIVE"
tar -tzf "$ARCHIVE" > "$OUT/archive-members.txt"

# Отдельно сверить members с заранее созданным manifest:
# size, sha256 каждого файла, mode/uid/gid, symlink target,
# xattrs (если применимо), пустые каталоги и отсутствие лишних путей.
# Не выполнять извлечение архива с непроверенными путями.
```

Сначала проверить, что архив **успешно записан и устойчив к внезапному завершению**, затем сверить точный набор файлов и совместимость читателей. Лишь после адресного решения RETIRED допускается освободить оригиналы. Оставшиеся защищённые/живые элементы внутри того же родителя исключаются из операции.

10.4. Чек-лист точечного удаления или архивирования

До изменения	После изменения
Device Online и тот же хост	Тот же хост/файловая система

До изменения	После изменения
Конкретный путь и владелец установлен	Исчез **только** согласованный путь
Нет in-flight/UNKNOWN, lock, текущего/будущего потребителя	Сервисный PID/состояние не ухудшились
Есть точный SHA/HEAD, восстановление, достаточный резерв	Источник восстановления и SHA сохранились
Проверены symlink, hardlink, uid/gid, xattrs, inode	Сохранены нужные внешние ссылки и права
Есть df/inode «до», идентификатор партии	Есть df/inode «после» и независимый readback
Отдельное разрешение на необратимое действие	Квитанция COMPLETE только после подтверждённого результата

Для платформенно заблокированных операций, например DEP-01, **фактическое состояние «BLOCKED / not executed» обязательно сохранять**. Разрешение владельца не заменяет успешный запуск инструмента. Не менять способ вызова специально для обхода действующего отказа.

10.5. Формат квитанции каждой партии

В уже существующем checkpoint/receipts (без создания нового daemon или реестра) сохранять как минимум:

```
operation_id, timestamp_utc, device_id,
owner_authorization, actual_author_ack,
paths_removed_or_compacted[], path_dev_inode_before[],
original_sha256[], recovery_reference[], recovery_sha256[],
head_and_lockfile[], in_flight_check[], ignored_wip_check[],
filesystem_available_before, filesystem_available_after,
inodes_before, inodes_after, own_allocated_blocks_removed,
service_readback[], linked_outputs_readback[],
status: PLANNED | BLOCKED | COMPLETE | FAILED,
unknowns[], next_owner_action
```

Не записывать в квитанции значения токенов/ключей, cookie, содержимое приватных профилей или данные маркетплейсов. Для нескольких параллельных задач отличать собственный объём физически удалённых блоков от общего изменения `df` сервера.

---

11. Матрица рисков и защитные решения

Риск	Вероятное последствие	Защита / решение	Кому принадлежит
Удалить незафиксированный WIP	Потеря изменений и уникальных тестов	Точный HEAD/patch/untracked/ignored /restore, проверка автора	Автор A/B/C
Вырезать shared inode по неверному du	Экономия окажется меньше плана	st_dev/st_ino/st_nlink, учёт внешних ссылок	Дисковый исполнитель
Удалить завершённый, но ожидаемый результат	Провал следующего теста или аудита	Согласовать будущих читателей и пути восстановления	Контроллер/Тестер
Убрать .next готового сервиса	Не стартует продукт или откат	Проверять RELEASE_SHA256SUMS; отдельно .next/cache	Владелец build/release
Принять разные Chrome-копии за взаимозаменяемые	Ошибка прав/безопасности/старта	Сохранять uid/gid/mode; не делать symlink между контекстами без проверки	Владелец browser-тестов

Риск	Вероятное последствие	Защита / решение	Кому принадлежит
Сжать журнал, который читают по старому пути	Поломка инструмента, потеря наблюдаемости	Reader gate и восстановление исходного пути	Владелец журнала
Сложить parent/child и одинаковые inode	Фиктивные 25 ГБ на бумаге	Единый список непересекающихся кандидатов, физический readback	Контроллер
Создать ещё один полный build ради проверки оптимизации	Временно съесть последние свободные ГБ	Проверять на следующем нужном продуктовом запуске, preflight	Автор сборки
Удалить rollback «не используется сейчас»	Невозможность отката	Проверить все pinned units/точки возврата, KEEP по умолчанию	Контроллер/оператор
Обойти отказ инструмента альтернативой	Нарушение контроля/непредсказуемое исполнение	Фиксировать отказ, исследовать причину штатно, не делать replay UNKNOWN	Исполнитель

Нормальное техническое решение при неопределённости — точная метка KEEP/WAIT с причиной и следующей проверкой, а не вечный общий запрет и не безусловное стирание.

---

12. Что считать реальным завершением программы

12.1. Приёмка места

1. Составлен перечень завершённых партий с точными путями, хешами восстановления и временем.
2. Указана **фактическая** сумма подтверждённых физических освобождений, а не итог условной модели; история параллельных записей сервера описана отдельно.
3. Сохранён минимум 3 072 МиБ свободного резерва и 200 000 inode **после вычета реальных будущих обязательств**.
4. Отдельно показано, достигнуто ли сокращение **25 000 000 000 байт**; если нет, честно зафиксирован остаток и доказанная причина KEEP.
5. Тестеру передан пиковый бюджет *всего следующего блока* с сохранением результатов, включая точное действительное число резервов. Передача ≠ фактическое выполнение блока.

12.2. Приёмка продукта и сохранности

- Исходные ветки, commit и уникальные WIP воспроизводятся.
- Нужные ZIP и CI/evidence доступны по SHA, аудит читает свои результаты.
- Все обязательные службы продолжают работать, pinned releases и rollback остались доступными.
- Ни один браузерный профиль, секрет, база, R3 и Business Bridge не изменены.
- Не появилось нового параллельного автора/уборщика и не повторена UNKNOWN/blocked операция.

12.3. Приёмка предотвращения нового роста

- Используемый генератор сборок сначала проверяет lockfile, версии и offline inputs, и только затем выделяет тяжёлое место.
- Повтор тех же неизменяемых входов не создаёт ещё одну полную физическую копию без причины, зафиксированной в checkpoint.
- После завершения одной реально необходимой сборки подтверждены **до / пик / после**, корректность пакета и изоляция изменяемых тестов.
- При новых проверках сохраняется **один канонический пакет** на подтверждённые входы, дополнительные результаты ссылаются на SHA вместо записи одинаковых ZIP.

- Контроллер закрывает временные пути по фактическому завершению, сохраняя WIP и архивы; периодически сравнивает прирост с запусками, а не только общим du.

Приложение A. Индекс первичных доказательств

Все машинные файлы находятся на Easyscript под:

/root/octoport-control/temporary/disk-owner/

Файл	Назначение	SHA-256
DISK_OWNER_20261031_OPTIMIZATION25_SUMMARY.json	Сводная модель, 128 адресных worktree-кандидатов, 254 KEEP-derived	`ceb2029a2cbbfdc0665350f7e6182cf9b0d1d0ec460e119a6b35069c0807cfc5`
DISK_OWNER_20261031_FULL_FOOTPRINT25.json	Основной единый footprint корневых каталогов	`0910644a9d5f789123870888d64353d3caae1bc13b869280ae28cbcbe8a3d305`
DISK_OWNER_20261031_CHECKOUT_FOOTPRINT25.json	Tracked-доли 364 worktree	`efa080e8f4a9254f5674c0fc4a928996714d89fbb5ffd7ac7b1ce348a7c5009d`
DISK_OWNER_20261031_WIP_COST25.json	Размер binary diff/untracked 221 старой копии	`a94cf411a47a7ba53599a0e8ec496888d3b8d5bbe5571489fb1c2b6d6016e80a`
DISK_OWNER_20261031_DERIVED_FOOTPRINT25.json	node_modules, .next, кэши и внешние links	`57cc416f4ef45867831bee48f146004267c32e49f1c8f95a8b26a8002ae353ef`
DISK_OWNER_20261031_HISTORY_COMPACTION25.json	Реальный в памяти tar+gzip/roundtrip для исторических файлов	`44d953f063cd685822654faeebba0acd23bedc5afcce566ca1183310cd0fa904`
DISK_OWNER_20261031_CHECKOUT_TOTALS25.json	Общие footprint старых деревьев и остаток	`11b8d044a808c4a85ca086a3ae9ff8dfa1b90b0d1e8d37be7a8128d7ade09f10`
DISK_OWNER_20261031_RELEASE_RETENTION25.json	Удерживаемые релизы и модель сжатия без замены	`61211f9bf8a3be53b6e8f3201e0f4602bf34cfb0de3268fdf891678ed338d3e0`
DISK_OWNER_20261031_R142_DEPENDENCY_RECOVERY.json	Проверенное восстановление трёх DEP-01 по донорам и 74 отличающимся файлам	`0c439ffa2c5fdafe8323ab3dda13992125941397d67b2b9620d48b01560750ae`

Примечание к использованию: наличие файла-доказательства на сервере не является разрешением менять соответствующий рабочий каталог. Аудит фиксировал одни снимки, к моменту исполнения размеры и состояния могут измениться.

Приложение B. Документы и точки координации

- Полный доказательный footprint и модель 25 ГБ — GitHub, commit 485bd28b.
- Дополнительный аудит дубликатов, CI-stage и браузерных бинарников — commit db5a8a52.
- Проверки восстановления DEP-01 и учёт запроса удаления — commit 488ecb40.
- Координация исходных авторов и возврата рабочего бюджета, PR354.

- Локальные обязательные правила: /root/octoport-control/current-prompts/DISK_LIFECYCLE.md, RESOURCE_POLICY.md и документы lifecycle рабочих копий. Последующие адресные указания владельца применяются с сохранением уже существующих запретов.

Приложение C. Краткий словарь для владельца

Git worktree: дополнительная распакованная папка одной версии проекта; сама по себе почти целиком повторяет код, уже хранящийся в Git. **WIP:** изменения, не доведённые до подтверждённого сохранения/публикации. **Untracked / ignored:** файлы, которые Git не хранит автоматически. **Hardlink:** два имени одного физического файла — удаление одного имени может не освободить ни байта. **node_modules:** установленные библиотеки, не основной продукт. **.next:** результаты Next.js, часть из них — нужная готовая сборка. **Receipt:** квитанция с точными фактами проверки/результата. **SHA-256:** контрольная сумма, показывающая, совпадают ли байты. **Rollback:** сохранённый путь возврата к прежнему работоспособному состоянию.

Заключение

Проект действительно не обязан постоянно занимать 50+ ГБ. Проверка показала техническую причину: сотни старых развернутых деревьев, многократные физические копии зависимостей и длительное хранение полных результатов вместо компактных доказательств. Главный возможный эффект дают **закрытие нетекущих worktree (11,730 ГБ)** и **компактное сохранение исторических результатов (5,765 ГБ)**; вместе с зависимостями и проверенными одиночными партиями базовая модель достигает **22,736 ГБ**.

Что необходимо для реального результата: назначить единый порядок координации прежних авторов, завершить адресные партии с проверкой восстановления и df после каждой, исправить используемый lifecycle сборки и новых пакетов. Не считать описание плана его выполнением, не удалять защищённые данные и не подгонять цель 25 ГБ арифметикой повторного счёта.